

Lab Assignment # 10

Course Title : AI Assistant Coding

Name of Student : M. Sai Pallavi

Enrollment No. : 2303A54063

Batch No. :48

Lab 10 – Code Review and Quality: Using AI to improve code quality and readability

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

• Screenshot of Generated Code:

```
LAB10_3.PY > ...
1 #TASK1: AI-Assisted Bug Detection
2 #The following Python function wrote by junior developer to calculate factorials:
3 """def factorial(n):
4     result = 1
5     for i in range(1, n):
6         result = result * i
7     return result"""
8 #Instructions:1. Run the code and test it with factorial(5),2. Use an AI assistant to: o I
9 #3. Compare the AI's corrected code with your own manual fix., 4. Write a brief comparison
10 #Expected Output: Corrected function should return 120 for factorial(5).
11 #Testing the original code
12 def factorial(n):
13     result = 1
14     for i in range(1, n):
15         result = result * i
16     return result
17 print(factorial(5)) # Output will be 24, which is incorrect
18 #AI-Assisted Bug Detection
19 #The logical bug in the code is that the loop runs from 1 to n-1, which means it does not
20 #Corrected version of the code
21 def factorial(n):
22     result = 1
23     for i in range(1, n + 1): # Change the loop to include n
24         result = result * i
25     return result
26 print(factorial(5)) # Output will be 120, which is correct
27 #Comparison:
28 #The AI correctly identified the off-by-one error and provided a corrected version of the
29 # However, both the original and corrected versions do not handle edge cases such as negat
30 #Improved version with edge case handling
31 def factorial(n):
32     if n < 0:
33         raise ValueError("Factorial is not defined for negative numbers.")
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/LAB10_3.PY"
24
120
120
1
Factorial is not defined for negative numbers.
```

Problem Statement 2: Improving Readability & Documentation

Scenario:The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Instructions:

5. Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.
- o Rewrite the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).

3. Exception handling for division by zero.
4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output: A well-documented, robust, and readable function that handles errors gracefully.

• **Screenshot of Generated Code:**

```
LAB10_3.PY > ...
48 #task2: Improving Readability & Documentation
49 #following code is poorly written but works:
50 """def calc(a, b, c):
51     if c == "add":
52         return a + b
53     elif c == "sub":
54         return a - b
55     elif c == "mul":
56         return a * b
57     elif c == "div":"""
58 #Instructions:
59 #5. Use AI to:
60 #-Critique the function's readability, parameter naming, and lack of documentation.
61 #-Rewrite the function with:
62 #1. Descriptive function and parameter names.
63 #2. A complete docstring (description, parameters, return value, examples).
64 #3. Exception handling for division by zero.
65 #4. Consideration of input validation.
66 #6. Compare the original and AI-improved versions.
67 #7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).
68 #Expected Output: A well-documented, robust, and readable function that handles errors gracefully.
69 #test the original code
70 def calc(a, b, c):
71     if c == "add":
72         return a + b
73     elif c == "sub":
74         return a - b
75     elif c == "mul":
76         return a * b
77     elif c == "div":
78         return a / b
79 print(calc(10, 5, "add")) # Output will be 15
80 print(calc(10, 5, "sub")) # Output will be 5
81 print(calc(10, 5, "mul")) # Output will be 50
82 print(calc(10, 5, "div")) # Output will be 2.
83 #AI-Improved version of the code
84 def calculate(num1, num2, operation):
85     """
86     Performs a specified arithmetic operation on two numbers.
87
88     Args:
89         num1 (float): The first number.
```

```

121     print(calculate(10, 5, "mod")) # This will raise an error
122 except ValueError as e:
123     print(e) # Output will be "Invalid operation. Supported operations are: add, subtract,
124 #Comparison:
125 #The original code lacked readability due to non-descriptive parameter names and no document
126 #It also did not handle edge cases such as division by zero or invalid operations. The AI-i
127 #Improved version with edge case handling
128 def calculate(num1, num2, operation):
129     """
130     Performs a specified arithmetic operation on two numbers.
131
132     Args:
133         num1 (float): The first number.
134         num2 (float): The second number.
135         operation (str): The operation to perform. Must be one of "add", "subtract", "multi
136
137     Returns:
138         float: The result of the arithmetic operation.
139
140     Raises:
141         ValueError: If the operation is not recognized or if division by zero is attempted.
142     """
143     if operation == "add":
144         return num1 + num2
145     elif operation == "subtract":
146         return num1 - num2
147     elif operation == "multiply":
148         return num1 * num2
149     elif operation == "divide":
150         if num2 == 0:
151             raise ValueError("Cannot divide by zero.")
152         return num1 / num2
153     else:
154         raise ValueError("Invalid operation. Supported operations are: add, subtract, multi

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/LAB10_3.PY"
50
2.0
Cannot divide by zero.
Invalid operation. Supported operations are: add, subtract, multiply, divide.

```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

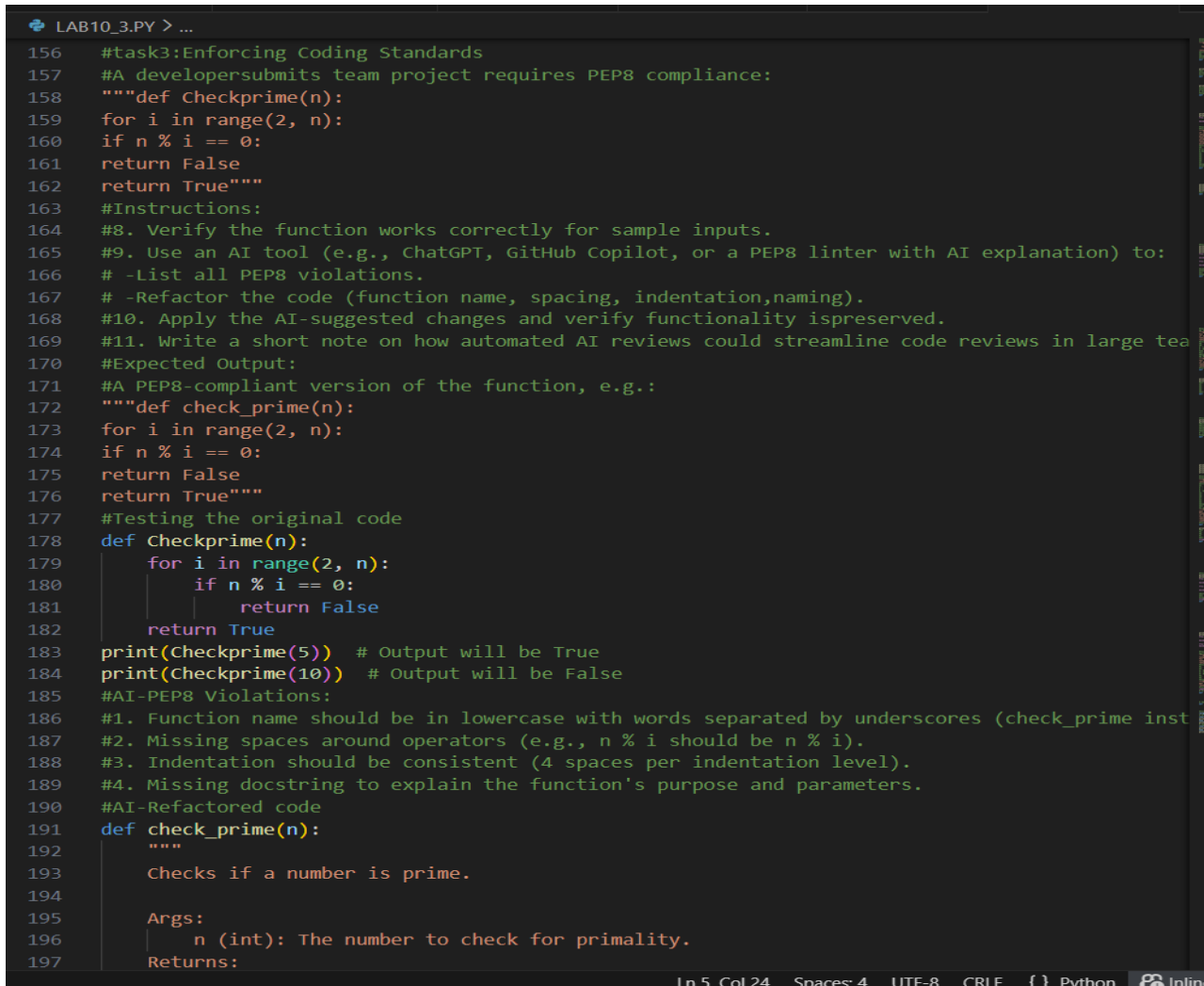
Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False
```

return True

• Screenshot of Generated Code:



```
LAB10_3.PY > ...  
156 #task3:Enforcing Coding Standards  
157 #A developersubmits team project requires PEP8 compliance:  
158 """def Checkprime(n):  
159     for i in range(2, n):  
160         if n % i == 0:  
161             return False  
162     return True"""  
163 #Instructions:  
164 #8. Verify the function works correctly for sample inputs.  
165 #9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:  
166 # -List all PEP8 violations.  
167 # -Refactor the code (function name, spacing, indentation,naming).  
168 #10. Apply the AI-suggested changes and verify functionality is preserved.  
169 #11. Write a short note on how automated AI reviews could streamline code reviews in large tea  
170 #Expected Output:  
171 #A PEP8-compliant version of the function, e.g.:  
172 """def check_prime(n):  
173     for i in range(2, n):  
174         if n % i == 0:  
175             return False  
176     return True"""  
177 #Testing the original code  
178 def Checkprime(n):  
179     for i in range(2, n):  
180         if n % i == 0:  
181             return False  
182     return True  
183 print(Checkprime(5)) # Output will be True  
184 print(Checkprime(10)) # Output will be False  
185 #AI-PEP8 Violations:  
186 #1. Function name should be in lowercase with words separated by underscores (check_prime inst  
187 #2. Missing spaces around operators (e.g., n % i should be n % i).  
188 #3. Indentation should be consistent (4 spaces per indentation level).  
189 #4. Missing docstring to explain the function's purpose and parameters.  
190 #AI-Refactored code  
191 def check_prime(n):  
192     """  
193     Checks if a number is prime.  
194     Args:  
195     n (int): The number to check for primality.  
196     Returns:
```

```

197     Returns:
198         bool: True if n is prime, False otherwise.
199     """
200     for i in range(2, n):
201         if n % i == 0:
202             return False
203     return True
204 #Testing the AI-refactored code
205 print(check_prime(5)) # Output will be True
206 print(check_prime(10)) # Output will be False
207 #Note on AI in code reviews:
208 #Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying
209 #and providing suggestions for improvement. This can save time for human reviewers, allowing them to focus on
210 #Comparison:
211 #The original code had several PEP8 violations, including non-compliant function naming, lack of
212 #inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues,
213 #improved version with edge case handling
214 def check_prime(n):
215     """
216     Checks if a number is prime.
217
218     Args:
219         n (int): The number to check for primality.
220     Returns:
221         bool: True if n is prime, False otherwise.
222     """
223     if n <= 1:
224         return False
225     for i in range(2, int(n**0.5) + 1):
226         if n % i == 0:
227             return False
228     return True
229 print(check_prime(5)) # Output will be True
230 print(check_prime(10)) # Output will be False
231 print(check_prime(1)) # Output will be False
232 print(check_prime(0)) # Output will be False
233 print(check_prime(-5)) # Output will be False
234

```

Ln 5, Col 24 Spaces: 4 UTF-8 CRLF {} Python

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/LAB10_3.PY"
True
False
True
False
True
False
False
False
False
False

```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario: In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - o Readability and naming.
 - o Reusability and modularity.
 - o Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output: An improved function with type hints, validation, and clearer intent,

e.g.: from typing import List, Union

```
def double_even_numbers(numbers: List[Union[int,  
float]]) -> List[Union[int, float]]:  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in numbers if isinstance(num,  
(int, float)) and num % 2 == 0]
```

• Screenshot of Generated Code

```
LAB10_3.PY > ...  
235 #task4:AI as a Code Reviewer in Real Projects  
236 #teammate submits a GitHub project  
237 """def processData(d):  
238     return [x * 2 for x in d if x % 2 == 0]"""  
239 #Instructions:  
240 #1. Manually review the function for:  
241 # - Readability and naming.  
242 # - Reusability and modularity.  
243 # -Edge cases (non-list input, empty list, non-integer elements).  
244 #2. Use AI to generate a code review covering:  
245 #a. Better naming and function purpose clarity.  
246 #b. Input validation and type hints.  
247 #c. Suggestions for generalization (e.g., configurable multiplier).  
248 #3. Refactor the function based on AI feedback.  
249 #4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.  
250 #Expected output:  
251 #An improved function with type hints, validation, and clearer intent,  
252 #e.g.:  
253 """from typing import List, Union  
254 def double_even_numbers(numbers: List[Union[int,  
255     float]]) -> List[Union[int, float]]:  
256     if not isinstance(numbers, list):  
257         raise TypeError("Input must be a list")  
258     return [num * 2 for num in numbers if isinstance(num,  
259         (int, float)) and num % 2 == 0]"""  
260 #Testing the original code  
261 def processData(d):  
262     return [x * 2 for x in d if x % 2 == 0]  
263 print(processData([1, 2, 3, 4, 5])) # Output will be [4, 8]  
264 #AI Code Review:  
265 #a. The function name `processData` is vague and does not clearly indicate its purpose. A better name would be `double_even_numbers`.  
266 #b. The function lacks input validation and type hints. It should check if the input is a list and use type hints for better readability and error handling.  
267 #c. The function could be generalized to allow for a configurable multiplier instead of hardcoding 2.  
268 #AI-Refactored code  
269 from typing import List, Union  
270 def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2):  
271     """  
272     Doubles the even numbers in a list using a specified multiplier.  
273     """  
274     Args:  
275         numbers (List[Union[int, float]]): A list of numbers to process.  
276         multiplier (Union[int, float], optional): The value to multiply the even numbers by. Defaults to 2.  
277     Returns:  
278         List[Union[int, float]]: A list of the processed numbers.
```

```
LAB10_3.PY > ...
298 """
299 #AI can be a valuable assistant in code reviews by quickly identifying issues related to rea
300 #improved version with edge case handling
301 from typing import List, Union
302 def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2)
303 """
304     Doubles the even numbers in a list using a specified multiplier.
305
306     Args:
307         numbers (List[Union[int, float]]): A list of numbers to process.
308         multiplier (Union[int, float], optional): The value to multiply the even numbers by.
309
310     Returns:
311         List[Union[int, float]]: A list of processed numbers where even numbers are multipli
312
313     Raises:
314         TypeError: If the input is not a list or if elements are not integers or floats.
315 """
316 if not isinstance(numbers, list):
317     raise TypeError("Input must be a list")
318 return [num * multiplier for num in numbers if isinstance(num, (int, float)) and num % 2
319 print(double_even_numbers([1, 2, 3, 4, 5])) # Output will be [4, 8]
320 print(double_even_numbers([1, 2, 3, 4, 5], multiplier=3)) # Output will be [6, 12]
321 try:
322     print(double_even_numbers("not a list")) # This will raise an error
323 except TypeError as e:
324     print(e) # Output will be "Input must be a list"
325 try:
326     print(double_even_numbers([1, 2, "three", 4, 5])) # This will raise an error
327 except TypeError as e:
328     print(e) # Output will be "Input must be a list"
329
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/LAB10_3.PY"
[4, 8]
[4, 8]
[6, 12]
Input must be a list
[4, 8]
[4, 8]
[6, 12]
Input must be a list
[4, 8]
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in

functions, vectorization with NumPy if applicable).

- o Provide an optimized version.

3. Compare execution time before and after optimization.

4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
```

```
return sum(x * x for x in numbers)
```

• Screenshot of Generated Code:

```
LAB10_3.PY > ...
329 #task5:AI-Assisted Performance Optimization
330 #given a function that processes a list of integers, but it runs slowly on large datasets:
331 """def sum_of_squares(numbers):
332     total = 0
333     for num in numbers:
334         total += num ** 2
335     return total"""
336 #Instructions:
337 #1. Test the function with a large list (e.g., range(1000000)).
338 #2. Use AI to:
339 #- Analyze time complexity.
340 #- Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy).
341 #- Provide an optimized version.
342 #3. Compare execution time before and after optimization.
343 #4. Discuss trade-offs between readability and performance.
344 #Expected Output:
345 """An optimized function, such as:
346 def sum_of_squares_optimized(numbers):
347     return sum(x * x for x in numbers)"""
348 #Testing the original code
349 import time
350 # Original Function
351 def sum_of_squares(numbers):
352     total = 0
353     for num in numbers:
354         total += num ** 2
355     return total
356 # Optimized Function (Correct Final Version)
357 def sum_of_squares_optimized(numbers):
358     if not hasattr(numbers, "__iter__"):
359         raise TypeError("Input must be an iterable of numbers.")
360     return sum(x * x for x in numbers)
361 #Testing
362 large_data = range(1000000)
363 # Original Version Timing
364 start_time = time.time()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
o Original Result: 333332833333500000
Execution time for original function: 0.04675006866455078 seconds
Optimized Result: 333332833333500000
Execution time for original function: 0.04675006866455078 seconds
Ln 5, Col 24 Spaces: 4 UTF-8 CRLF { } Python
```

